

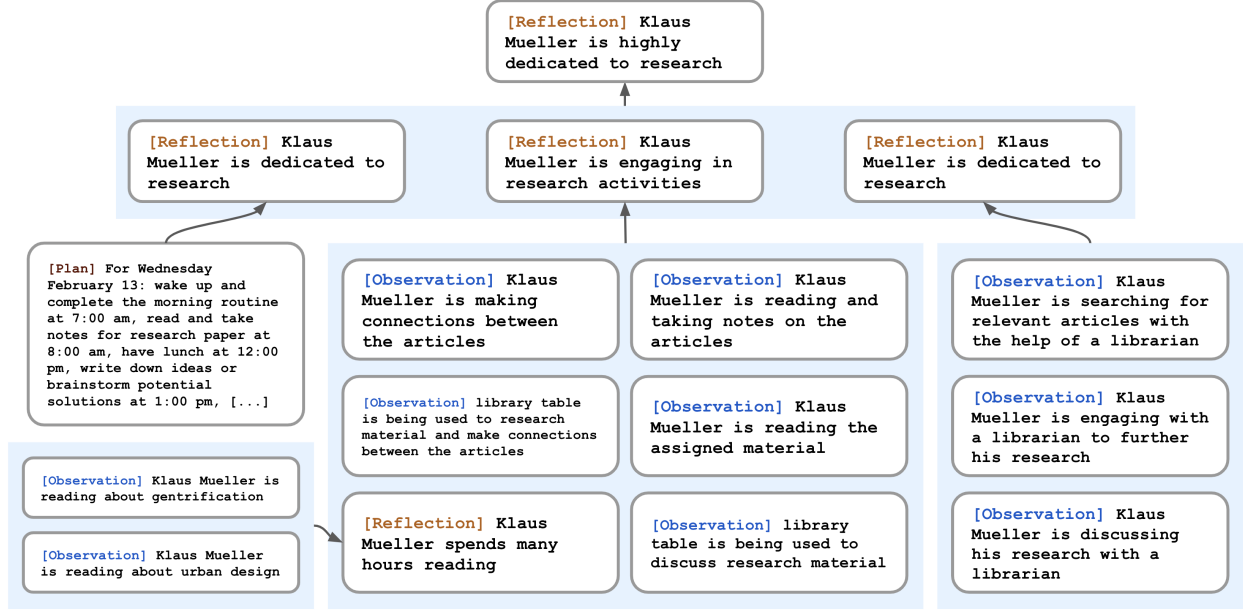


Figure 7: A reflection tree for Klaus Mueller. The agent’s observations of the world, represented in the leaf nodes, are recursively synthesized to derive Klaus’s self-notion that he is highly dedicated to his research.

recognize Maria putting in effort into her own research (albeit in a different field), enabling a reflection that they share a common interest. With the approach below, when Klaus is asked who to spend time with, Klaus chooses Maria instead of Wolfgang.

Approach: We introduce a second type of memory, which we call a *reflection*. Reflections are higher-level, more abstract thoughts generated by the agent. Because they are a type of memory, they are included alongside other observations when retrieval occurs. Reflections are generated periodically; in our implementation, we generate reflections when the sum of the importance scores for the latest events perceived by the agents exceeds a threshold (150 in our implementation). In practice, our agents reflected roughly two or three times a day.

The first step in reflection is for the agent to determine what to reflect on, by identifying questions that can be asked given the agent’s recent experiences. We query the large language model with the 100 most recent records in the agent’s memory stream (e.g., “Klaus Mueller is reading a book on gentrification”, “Klaus Mueller is conversing with a librarian about his research project”, “desk at the library is currently unoccupied”) and prompt the language model, “Given only the information above, what are 3 most salient high-level questions we can answer about the subjects in the statements?” The model’s response generates candidate questions: for example, What topic is Klaus Mueller passionate about? and What is the relationship between Klaus Mueller and Maria Lopez? We use these generated questions as queries for retrieval, and gather relevant memories (including other reflections) for each question. Then we prompt the language model to extract insights and cite the particular records that served as evidence for the insights. The full prompt is as follows:

Statements about Klaus Mueller

1. Klaus Mueller is writing a research paper
 2. Klaus Mueller enjoys reading a book on gentrification
 3. Klaus Mueller is conversing with Ayesha Khan about exercising [...]
- What 5 high-level insights can you infer from the above statements? (example format: insight (because of 1, 5, 3))

This process generates statements such as Klaus Mueller is dedicated to his research on gentrification (because of 1, 2, 8, 15). We parse and store the statement as a reflection in the memory stream, including pointers to the memory objects that were cited.

Reflection explicitly allows the agents to reflect not only on their observations but also on other reflections: for example, the second statement about Klaus Mueller above is a reflection that Klaus previously had, not an observation from his environment. As a result, agents generate trees of reflections: the leaf nodes of the tree represent the base observations, and the non-leaf nodes represent thoughts that become more abstract and higher-level the higher up the tree they are.

4.3 Planning and Reacting

Challenge: While a large language model can generate plausible behavior in response to situational information (e.g., [46, 80]), agents need to plan over a longer time horizon to ensure that their sequence of actions is coherent and believable. If we prompt a language model with Klaus’s background, describe the time, and ask what action he ought to take at the given moment, Klaus would eat lunch at 12 pm, but then again at 12:30 pm and 1 pm, despite having already

eaten his lunch twice. Optimizing for believability in the moment sacrifices believability over time. To overcome this issue, planning is essential. With the approach described below, Klaus’s afternoon plan is less gluttonous: he has lunch at Hobbs Cafe while reading at 12pm, works on his research paper at the school library at 1pm, and takes a break for a walk in the park at 3pm.

Approach: Plans describe a future sequence of actions for the agent, and help keep the agent’s behavior consistent over time. A plan includes a location, a starting time, and a duration. For instance, Klaus Mueller, who is dedicated in his research and has an impending deadline,⁵ may choose to spend his day working at his desk drafting his research paper. An entry in a plan might state, for example: for 180 minutes from 9am, February 12th, 2023, at Oak Hill College Dorm: Klaus Mueller’s room: desk, read and take notes for research paper. Like reflections, plans are stored in the memory stream and are included in the retrieval process. This allows the agent to consider observations, reflections, and plans all together when deciding how to behave. Agents may change their plans midstream if needed.

It would be unrealistic and uninteresting for an artist agent to plan on painting while sitting at a pharmacy counter for four hours without moving. A more desirable plan would involve the agent taking the necessary time to gather materials, mix paint, take breaks, and clean up during the four-hour period in their home studio. To create such plans, our approach starts top-down and then recursively generates more detail. The first step is to create a plan that outlines the day’s agenda in broad strokes. To create the initial plan, we prompt the language model with the agent’s summary description (e.g., name, traits, and a summary of their recent experiences) and a summary of their previous day. A full example prompt is below, which is unfinished at the bottom for the language model to complete:

Name: Eddy Lin (age: 19)
 Innate traits: friendly, outgoing, hospitable
 Eddy Lin is a student at Oak Hill College studying music theory and composition. He loves to explore different musical styles and is always looking for ways to expand his knowledge. Eddy Lin is working on a composition project for his college class. He is taking classes to learn more about music theory. Eddy Lin is excited about the new composition he is working on but he wants to dedicate more hours in the day to work on it in the coming days
 On Tuesday February 12, Eddy 1) woke up and completed the morning routine at 7:00 am, [. . .]
 6) got ready to sleep around 10 pm.
 Today is Wednesday February 13. Here is Eddy’s plan today in broad strokes: 1)

This generates a rough sketch of the agent’s plan for a day, divided into five to eight chunks: “1) wake up and complete the morning routine at 8:00 am, 2) go to Oak Hill College to take classes starting 10:00 am, [. . .] 5) work on his new music composition from 1:00 pm to 5:00 pm, 6) have dinner at 5:30 pm, 7) finish school assignments and go to bed by 11:00 pm.”

⁵And, in this way, bears at least a passing resemblance to the authors of this paper.

The agent saves this plan in the memory stream and then recursively decomposes it to create finer-grained actions, first into hour-long chunks of actions—Eddy’s plan to work on his new music composition from 1:00 pm to 5:00 pm becomes 1:00 pm: start by brainstorming some ideas for his music composition [...] 4:00 pm: take a quick break and recharge his creative energy before reviewing and polishing his composition. We then recursively decompose this again into 5–15 minute chunks: e.g., 4:00 pm: grab a light snack, such as a piece of fruit, a granola bar, or some nuts. 4:05 pm: take a short walk around his workspace [...] 4:50 pm: take a few minutes to clean up his workspace. This process can be adjusted to match the desired granularity.

4.3.1 Reacting and Updating Plans. Generative agents operate in an action loop where, at each time step, they perceive the world around them and those perceived observations are stored in their memory stream. We prompt the language model with these observations to decide whether the agent should continue with their existing plan, or react. Standing at an easel and painting, for example, might trigger an observation of the easel, but this is unlikely to prompt a reaction. However, if Eddy’s father John records that he sees Eddy taking a short walk in the house garden, the outcome is different. The prompt is below, with [Agent’s Summary Description] standing in for a dynamically-generated, paragraph-long summary of the agent’s overall goals and disposition, which is described in Appendix A:

[Agent’s Summary Description]
 It is February 13, 2023, 4:56 pm.
 John Lin’s status: John is back home early from work.
 Observation: John saw Eddy taking a short walk around his workplace.
 Summary of relevant context from John’s memory:
 Eddy Lin is John’s Lin’s son. Eddy Lin has been working on a music composition for his class. Eddy Lin likes to walk around the garden when he is thinking about or listening to music.
 Should John react to the observation, and if so, what would be an appropriate reaction?

The context summary is generated through two prompts that retrieve memories via the queries “What is [observer]’s relationship with the [observed entity]?” and “[Observed entity] is [action status of the observed entity]”, and their answers summarized together. The output suggests that John could consider asking Eddy about his music composition project. We then regenerate the agent’s existing plan starting from the time when the reaction takes place. Finally, if the action indicates an interaction between agents, we generate their dialogue.

4.3.2 Dialogue. Agents converse as they interact with each other. We generate agents’ dialogue by conditioning their utterances on their memories about each other. For example, when John initiates his conversation with Eddy, we generate John’s first utterance by using his summarized memory about Eddy and the intended reaction when he decided to ask Eddy about his composition project:

[Agent’s Summary Description]
 It is February 13, 2023, 4:56 pm.

John Lin's status: John is back home early from work.

Observation: John saw Eddy taking a short walk around his workplace.

Summary of relevant context from John's memory: Eddy Lin is John's son. Eddy Lin has been working on a music composition for his class. Eddy Lin likes to walk around the garden when he is thinking about or listening to music.

John is asking Eddy about his music composition project. What would he say to Eddy?

The result: "Hey Eddy, how's the music composition project for your class coming along?" From Eddy's perspective, John initiating the dialogue is seen as an event to which he may want to react. So, just as John did, Eddy retrieves and summarizes his memory about his relationship with John, as well as his memory that may be related to John's last utterance in the dialogue. If he decides to respond, we generate Eddy's utterance using his summarized memory and the current dialogue history:

[Agent's Summary Description]

It is February 13, 2023, 4:56 pm.

Eddy Lin's status: Eddy is taking a short walk around his workplace.

Observation: John is initiating a conversation with Eddy.

Summary of relevant context from Eddy's memory: John Lin is Eddy Lin's father. John Lin is caring and is interested to learn more about Eddy Lin's school work. John Lin knows that Eddy Lin is working on a music composition.

Here is the dialogue history:

John: Hey Eddy, how's the music composition project for your class coming along?

How would Eddy respond to John?

This generates Eddy's response: "Hey Dad, it's going well. I've been taking walks around the garden to clear my head and get some inspiration." The continuation of this dialogue is generated using the same mechanism until one of the two agents decides to end the dialogue.

5 SANDBOX ENVIRONMENT IMPLEMENTATION

The Smallville sandbox game environment is built using the Phaser web game development framework [57]. The visual environment sprites, including agent avatars, as well as an environment map and collision map that we authored, are imported into Phaser.

We supplement the sandbox development framework with a server that makes the sandbox information available to generative agents and enables generative agents to move and influence the sandbox environment. The server maintains a JSON data structure that contains information about each agent in the sandbox world, including their current location, a description of their current action, and the sandbox object they are interacting with. At each sandbox time step, the sandbox server parses the JSON for any changes coming from the generative agents, moves the agents to their new positions, and updates the status of any sandbox objects that the

agents are interacting with (e.g., changing the status of the coffee machine from "idle" to "brewing coffee" if an agent's action is "making espresso for a customer @ Hobbs Cafe: counter: coffee machine"). The sandbox server is also responsible for sending all agents and objects that are within a preset visual range for each agent to that agent's memory, so the agent can react appropriately. The agent's output action then updates the JSON, and the process loops for the next time step.

End users initialize a new agent with a brief natural language description, as in the paragraph about John Lin in Section 3.1. In our implementation, we split this semicolon-delimited list of characteristics up into a set of memories. These serve as the initial memories that determine the agent's behavior. These memories are initial starting points: as the agents gain more experience in the sandbox world, and as more records saturate the memory stream, the agent's summary and behavior will evolve.

5.1 From Structured World Environments to Natural Language, and Back Again

The architecture of generative agents operates using natural language. Therefore, we need a mechanism to ground the agent's reasoning to the sandbox world. To achieve this, we represent the sandbox environment—areas and objects—as a tree data structure, with an edge in the tree indicating a containment relationship in the sandbox world. We convert this tree into natural language to pass to the generative agents. For instance, "stove" being a child of "kitchen" is rendered into "there is a stove in the kitchen."

Agents build individual tree representations of the environment as they navigate it — subgraphs of the overall sandbox environment tree. We initialize each agent with an environment tree capturing the spaces and objects that the agent should be aware of: the rooms and objects in their living quarters, their workplace, and commonly visited stores and shops. As the agents navigate the sandbox world, they update this tree to reflect newly perceived areas. Agents are not omniscient: their tree may get out of date as they leave an area, and is updated when they re-enter the area.

To determine the appropriate location for each action, we traverse the agent's stored environment tree and flatten a portion of it into natural language to prompt the language model. Recursively starting at the root of the agent's environment tree, we prompt the model to find the most suitable area. For example, if Eddy's agent indicated that he should take a short walk around his workspace:

[Agent's Summary Description]

Eddy Lin is currently in The Lin family's house: Eddy Lin's bedroom: desk) that has Mei and John Lin's

bedroom, Eddy Lin's bedroom, common room, kitchen, bathroom, and garden.

Eddy Lin knows of the following areas: The Lin family's house, Johnson Park, Harvey Oak Supply Store, The Willows Market and Pharmacy, Hobbs Cafe, The Rose and Crown Pub.

* Prefer to stay in the current area if the activity can be done there.

Eddy Lin is planning to take a short walk around his workspace. Which area should Eddy Lin go to?